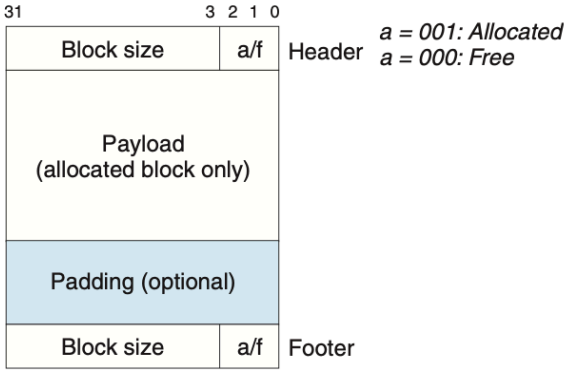
Malloc Lab 구현 기획문서

mm.c 파일만 수정하여 제출 필요함.
표준 라이브러리 함수나, 시스템 호출 사용하면 안 됨 (제공되는 지원 루틴 사용해야함)
배열, 구조체, 트리 등 전역 또는 정적(static) 복합 자료구조 정의할 수 없음
(단, 정수나 포인터 같은 전역 스칼라 변수는 허용)

1. 전체 설계 개요

- 블록 구조 그림 (header, payload, padding, footer)

Figure 9.39
Format of heap block that uses a boundary tag.



- 정렬 기준: 8 바이트
- 최소 블록 크기: 16 바이트 (이유: header(4B) + payload(1B이상) + padding + footer (4B))
- 워드 크기: 4 바이트
- 더블워드 크기: 8 바이트

2. 힙 초기 레이아웃

- 초기 힙 구조 그림 (padding, prologue header, prologue footer, epilogue header)

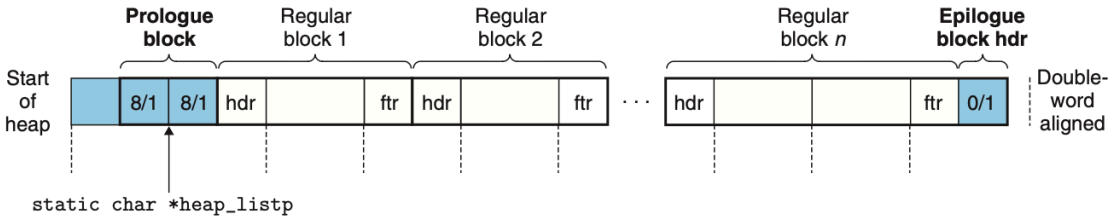


Figure 9.42 **Invariant form of the implicit free list.**

- **heap_listp**가 가리키는 위치: 프롤로그 블록 (최적화 시 프롤로그 다음 블록 가르키게)
- **prologue** 블록 역할: coalescing 할 때 첫 블록이냐 분기 안타게 함
- **epilogue** 블록 역할: coalescing 할 때 끝 블록이냐 분기 안타게 함

3. 매크로 역할 정리 **(**코드 아님**,** 역할만**)**

	매크로 이름	역할 **(**한 줄 설명**)**	비고
기본 **size** 함수	**WSIZE**	Word size(워드 크기), header/footer 사이즈	bytes 4bytes = 1word
	DSIZE	Double word size (더블 워드 크기)	bytes 8bytes = 2word
	CHUNKSIZE	초기 가용 블록과 힙 확장을 위한 기본 크기 2^12 = 4096 의미 (힙이 부족할 때 메모리 더 달라고 요청하는 단위)	bytes (바이트단위)
	MAX(x, y)	둘 중 큰 값을 반환	
가용 리스트에 접근하고 방문하는 작은 매크로 정의	**PACK(size, alloc)**	size와 alloc(할당비트)를 통합해서 header/footer 저장할 수 있는 값 리턴	두 값을 비트별로 OR 연산한다. 그러면 단 4바이트(1워드)만 사용해서 두 가지 정보를 동시에 저장할 수 있다.
	GET(p)	인자 p가 참조하는 워드를 읽어서 리턴	캐스팅이 매우 중요. p는 대개 (void *) 포인터이며, 직접적으로 역참조할 수는 없다.

			메모리 주소나 블록의 크기 정보는 음수가 될 수 없다. 하나의 워드는 4바이트이다.
	PUT(p, val)	인자 p가 가르키는 워드에 val을 저장	
	GET_SIZE(p)	p에 있는 헤더 또는 푸터의 size 반환	1) 하위 3비트를 강제로 0으로 만들어야 함. 2) 0x7:00…00111 3) ~: 비트 반전 4) &: 비트 and 연산
	GET_ALLOC(p)	p에 있는 헤더 또는 푸터의 할당 비트 반환	맨 끝 비트만 봐야함. 1이면 할당중 0이면 free
	HDRP(bp)	bp(block pointer)의 헤더를 가르키는 포인터를 리턴	bp는 payload의 시작점을 가르키고 있음 포인터에 숫자를 더하거나 빼면 포인터가 가르키는 타입의 크기 x 숫자 만큼 주소 이동.

	FTRP(bp)	bp(block pointer)의 푸터를 가르키는 포인터를 리턴	다음 블록의 페이로드에서 푸터로 접근해보자
	NEXT_BLKP(bp)	다음 블록 포인터를 리턴	헤더에서 사이즈를 읽어서 이동
	PREV_BLKP(bp)	이전 블록 포인터를 리턴	푸터에서 사이즈를 읽어서 이동

⚠️ 중요: bp는 블록의 페이로드 시작주소를 가리킨다.

4. 함수별 설계

mm_init

- 역할: 초기 힙 영역 할당 등 필요한 초기화 작업을 수행
- 하는 일 (순서대로):
  1. 4워드의 빈 영역 초기화
  2. 빈 가용 리스트 초기화, 패딩, 프롤로그 블록, 에필로그 블록, heap_listp 초기화
  3. 힙 페이지 늘리기 (CHUNKSIZE 바이트만큼)
- 호출하는 함수: mem_sbrk, extend_heap
- 반환값: 성공 시 0, 문제 발생 시 -1
- 실패 조건: 공간 할당이 실패할 경우 (빈 heap, heap 늘리기)

---

extend_heap(words)

- 역할: 새 가용 블록으로 힙 확장하기
- 언제 호출되는가:
  1. 힙이 초기화 될 때
  2. mm_malloc이 적당한 맞춤 fit을 찾지 못했을 때
- 하는 일 (순서대로):
  1. 요청한 크기를 인접 2워드의 배수(8byte)로 반올림, 메모리 시스템에 추가 공간 요청
  2. 새로 생긴 빈 공간에 헤더와 푸터 초기화, 맨 뒤에 에필로그 헤더 다시 만들기
  3. 바로 앞 블록이 비어있다면 coalesce 하기.
- 호출하는 함수: mem_sbrk, coalesce
- 반환값: 성공 시 coalesce의 반환값, 실패 시 NULL

---

mm_malloc(size)

- 역할**:** 최소 요청된 크기(size) 바이트만큼 할당된 메모리 블록의 포인터를 반환, 표준 C 라이브러리(libc)와 동일하게 항상 **8**바이트 단위로 정렬된**(8-byte aligned)** 포인터를 반환해야 한다.
- 입력**:** payload 크기
- 크기 계산 로직**:** 8바이트보다 작으면 16바이트로 만듬(최소 블록 크기), 8바이트보다 크면 일단 size에 오버헤드 확보(헤더/푸터 공간) 이후 8의 배수로 올림(rounding up) 계산식: (((기존 크기 + 오버헤드) + 배수 - 1)) / 배수) * 배수
- 하는 일 **(**순서대로**):**
  1. asize (Adjusted Size)로 가공하기
  2. find_fit
  3.
- 분기**:**
  - find_fit 성공 시: 반환된 bp에 asize만큼 할당한다
  - find_fit 실패 시: 새로운 힙 공간을 만들어 할당한다
- 반환값**: 8**바이트 단위로 정렬된**(8-byte aligned)** 포인터 반환

---

mm_free(bp)

- 역할**:** ptr이 가리키는 블록을 해제한다. 반환값 없음. 해제되지 않은 포인터(ptr)가 전달되었을 때만 작동 보장
- 입력**:** block pointer
- 하는 일 **(**순서대로**):**
  1. 블록의 사이즈를 받아, 헤더와 푸터의 alloc bit를 0으로 초기화
  2. coalesce
- 호출하는 함수**:** coalesce

---

coalesce(bp)

- 역할**:** 이전/다음 블록이 free 블록이라면 합치고 합쳐진 블록의 시작점을 반환
- 판단 기준**:** 이전/다음 블록의 allocate bit를 확인
- **4 Cases:**
  - Case 1 (이전 alloc, 다음 alloc): 그대로 반환
  - Case 2 (이전 alloc, 다음 free): 다음 블록의 사이즈를 합쳐서 헤더 푸터 초기화
  - Case 3 (이전 free, 다음 alloc): 이전 블록의 사이즈를 합쳐서 헤더 푸터 초기화, 이때 이전 블록에서부터 포인터 초기화
  - Case 4 (이전 free, 다음 free): 이전, 다음 블록 사이즈를 합쳐서 헤더 푸터 초기화, 이때 이전 블록에서부터 포인ㅇ터 초기화
- 각 **case**에서 갱신하는 것**:** header 위치, footer 위치, 크기
- 반환값**:** 주변의 가용 블록들을 합친 후, 합쳐진 가용 블록의 시작점(bp) 반환 static void * (왜? 현재 파일에서만 접근가능하게 함)

---

find_fit(asize)

- 역할: asize에 맞는 할당할 공간을 찾는다
- 탐색 시작점: 힙의 첫 블록(프롤로그 다음 첫 블록)부터
- 탐색 종료 조건: 마지막 블록까지 (에필로그 앞)
- 배치 정책: (first fit / next fit / best fit 중 선택하고 이유) first fit (기본구현은 first fit)
- 반환값:
    - 성공: static void *
    - 실패: NULL (no fit)

---

place(bp, asize)

- 역할: 헤더, 푸터 초기화
- 입력: 새로 할당할 block pointer, 할당할 asize
- **Splitting** 판단 기준: 남는 크기 >= 최소 블록 크기
- **Splitting** 할 때:
  1. 할당할 블럭의 헤더와 푸터를 alloc bit 1로 설정한다.
  2. free 블럭의 헤더와 푸터를 alloc bit 0으로 설정한다.
- **Splitting** 안 할 때:
  1. 해당 공간을 다 사용한다.

---

mm_realloc(ptr, size)

- 역할: ptr이 NULL이 아닌 경우, 반드시 이전의 malloc, reallc 호출에 의해 반환된 포인터여야함. ptr이 가르키는 블록(이전 블록)의 크기를 size 바이트로 변경하고 새 블록의 주소를 반환. 새 블록의 내용은 ptr 블록의 내용 중, 최소한 새로운 크기와 이전 크기 중 더 작은 값만큼은 유지되어야 함.
- 특수 케이스:
    - ptr == NULL: mm_malloc(size)와 동일
    - size == 0: mm_free(ptr)와 동일, NULL 반환
- 기본 구현 전략:
  1.새로 할당을 받는다.
  2. 기존 데이터를 복사한다. 만약 새로 받은 사이즈가 기존 데이터보다 작다면 새로 받은 사이즈 만큼만 memcpy로 복사를 해준다.
  3. 기존 블록을 해제하고 새로운 포인터를 반환한다.
- 최적화 아이디어: 1) 뒤에 가용블럭이 있다면 헤더를 업데이트해준다.

5. 전체 호출 흐름도

malloc 흐름:

mm_malloc(size) → 크기 계산 (정렬) → find_fit(asize) → 성공: place(bp, asize) → 실패: ___ → 반환: ___

free 흐름:

mm_free(bp) → 헤더 푸터 alloc bit 초기화 → coalesce(bp) → Case 판단 → 가용블록 시작주소 반환

init 흐름**:**

mm_init() → mem_sbrk(4*word) → implicit list 초기화→ extend_heap(words)

6. 점수 향상 로드맵 **(**나중에 채울 것**)**

단계	전략	예상 개선
1단계	implicit + first fit	기본 점수
2단계		
3단계		
4단계		

7. 메모 **/** 삽질 기록

(구현하면서 막힌 부분, 해결 방법 기록용)